
DATA VISUALIZATION

Libraries Documentation

Matplotlib & Seaborn

Data visualization is the representation of information in a graphical or pictorial format that enables better understanding, exploration, and communication of data. By transforming raw numbers into visual patterns, trends, and comparisons become immediately apparent — enabling both analysts and stakeholders to make faster, more informed decisions.

Python offers a rich ecosystem of visualization libraries. This guide provides comprehensive documentation for two of the most widely used: Matplotlib and Seaborn. Whether you are just beginning or looking to sharpen your skills, this resource covers introductions, graph types with code snippets, and an in-depth comparison.

1. LIBRARY INTRODUCTIONS

Before diving into specific graph types, it helps to understand what each library is designed for, where it excels, and what kind of audience it targets. Both Matplotlib and Seaborn have earned permanent places in the data scientist's toolkit, though they solve slightly different problems.

1.1 Matplotlib

Matplotlib was introduced in 2002 by John D. Hunter with the goal of producing publication-quality plots in Python — a language that, at the time, had no mature charting solution. It is built on NumPy arrays and exposes a MATLAB-inspired pyplot interface that makes simple charts trivial while still allowing fine-grained control over every pixel.

What sets Matplotlib apart?

Matplotlib is the foundational plotting library of the Python scientific ecosystem. Almost every other visualization library (Seaborn, Pandas .plot(), etc.) is built on top of it. Its object-oriented API gives you complete control over figures, axes, ticks, labels, and artists — making it the library of choice whenever you need precise, publication-ready output.

CORE COMPONENTS OF A MATPLOTLIB FIGURE

- **Figure** — The top-level container that holds one or more plots (Axes).
- **Axes** — The actual plot area (what you think of as "a chart"); each Figure can contain multiple Axes.

-
- **Axis** — Manages tick marks, tick labels, and the numerical range along a single dimension.
 - **Artists** — Everything visible on the figure (lines, text, patches, images) is an Artist.

MAIN APPLICATIONS

- Publication-quality static figures for papers and reports
- Fine-grained customisation of every chart element
- Subplots and complex multi-panel layouts
- Animated and interactive figures inside Jupyter notebooks
- Base layer for higher-level libraries such as Seaborn and Pandas

1.2 Seaborn

Seaborn is a statistical data visualization library built directly on top of Matplotlib. Created by Michael Waskom and first released in 2012, it aims to make creating informative, attractive statistical graphics as easy as possible. Where Matplotlib forces you to write many lines of code to produce a decent-looking chart, Seaborn provides high-level functions that deliver beautiful results with a single line.

What sets Seaborn apart?

Seaborn integrates tightly with Pandas DataFrames, speaks the language of statistics (distributions, confidence intervals, regression lines), and ships with carefully chosen default themes and color palettes. Its figure-level functions (`relplot`, `catplot`, `displot`) automatically handle faceting — splitting a chart into a grid of subplots based on a categorical variable — which would require many lines of boilerplate in pure Matplotlib.

PLOT CATEGORIES IN SEABORN

- **Relational Plots** — `scatterplot()`, `lineplot()`, `relplot()`
- **Categorical Plots** — `barplot()`, `countplot()`, `boxplot()`, `violinplot()`, `swarmplot()`, `pointplot()`, `catplot()`
- **Distribution Plots** — `histplot()`, `kdeplot()`, `rugplot()`, `displot()`
- **Regression Plots** — `regplot()`, `lmplot()`
- **Matrix Plots** — `heatmap()`, `clustermap()`

MAIN APPLICATIONS

- Exploratory data analysis (EDA) on Pandas DataFrames
- Statistical visualisation with built-in confidence intervals
- Multi-panel faceted plots with minimal code
- Correlation matrices and heatmaps
- Quick and beautiful charts for presentations and dashboards

2. MATPLOTLIB — GRAPH TYPES

Matplotlib's pyplot sub-module provides a comprehensive set of chart functions. Below you will find the most commonly used graph types, along with descriptions, use cases, and runnable code snippets.

2.1 Line Chart

DESCRIPTION

A line chart connects consecutive data points with a continuous line, making it ideal for showing trends over a continuous dimension such as time. The default `plt.plot()` function produces a line chart.

USE CASE

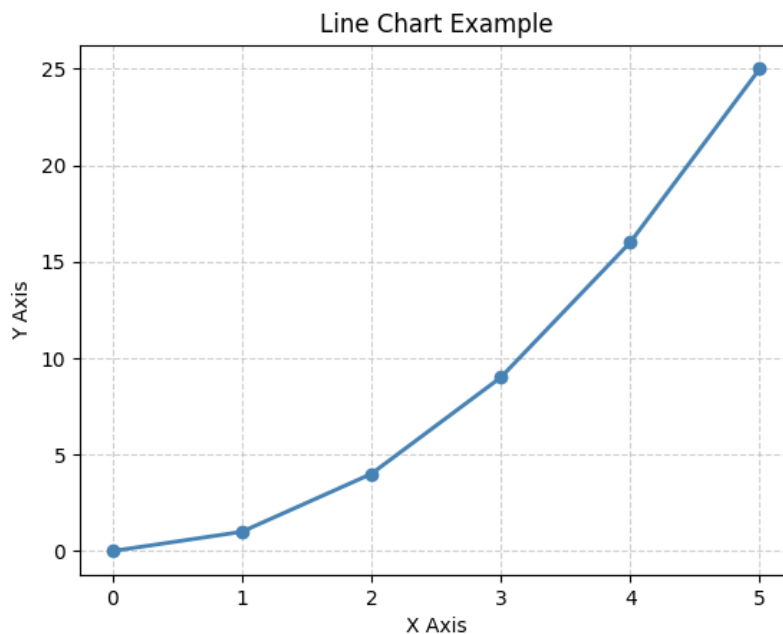
Use line charts when you need to show change or trend over time — stock prices, temperature readings, model training loss curves, or website traffic over a period.

CODE SNIPPET

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([0, 1, 2, 3, 4, 5])
y = np.array([0, 1, 4, 9, 16, 25])

plt.plot(x, y, color='steelblue', linewidth=2, marker='o')
plt.title('Line Chart Example')
plt.xlabel('X Axis')
plt.ylabel('Y Axis')
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```



DESCRIPTION OF PARAMETERS

- `np.array()` — Creates a NumPy array from the given list of values.
- `plt.plot(x, y)` — Plots `y` against `x` as a line; additional keyword arguments control style.
- `color`, `linewidth`, `marker` — Control line colour, thickness, and point markers.
- `plt.grid()` — Adds a background grid for easier reading.
- `plt.show()` — Renders and displays the figure.

2.2 Bar Chart

DESCRIPTION

A bar chart uses rectangular bars whose heights (or lengths) represent the value of each category. It is one of the most commonly used chart types for comparing discrete categories.

USE CASE

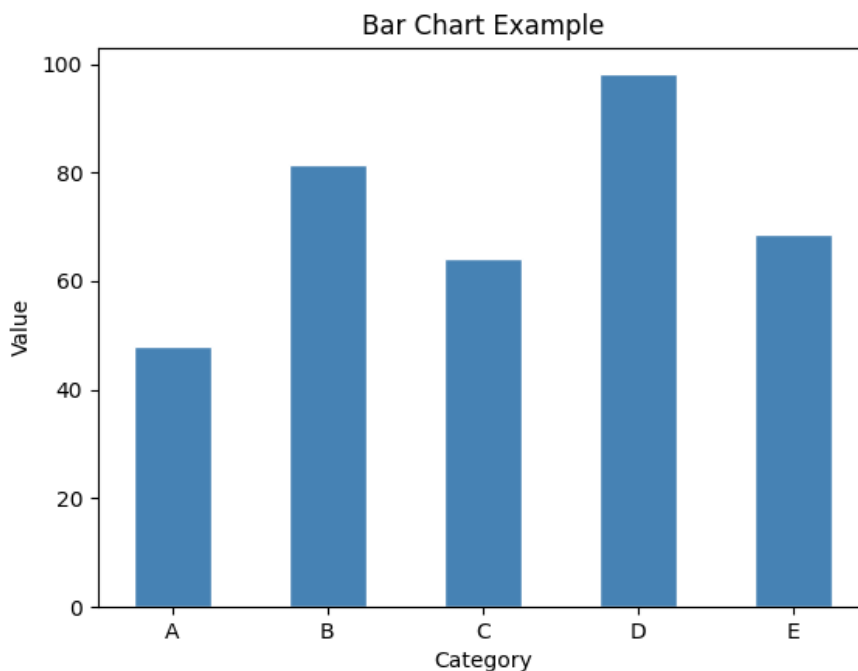
Use bar charts to compare quantities across distinct categories such as monthly sales by region, survey responses across age groups, or feature importance scores.

CODE SNIPPET

```
import matplotlib.pyplot as plt
import numpy as np

categories = ['A', 'B', 'C', 'D', 'E']
values = np.random.rand(5) * 100

plt.bar(categories, values, color='steelblue', width=0.5, edgecolor='white')
plt.title('Bar Chart Example')
plt.xlabel('Category')
plt.ylabel('Value')
plt.show()
```



DESCRIPTION OF PARAMETERS

- `np.random.rand(5)` — Generates five random floating-point values between 0 and 1.
- `plt.bar(x, height)` — Plots vertical bars; `x` can be labels or numeric positions.
- `width` — Controls the width of each rectangular bar (default 0.8).
- `color, edgecolor` — Fill colour and outline colour of bars.

2.3 Histogram

DESCRIPTION

A histogram displays the frequency distribution of a continuous variable by dividing the data into contiguous bins and counting how many observations fall into each bin.

USE CASE

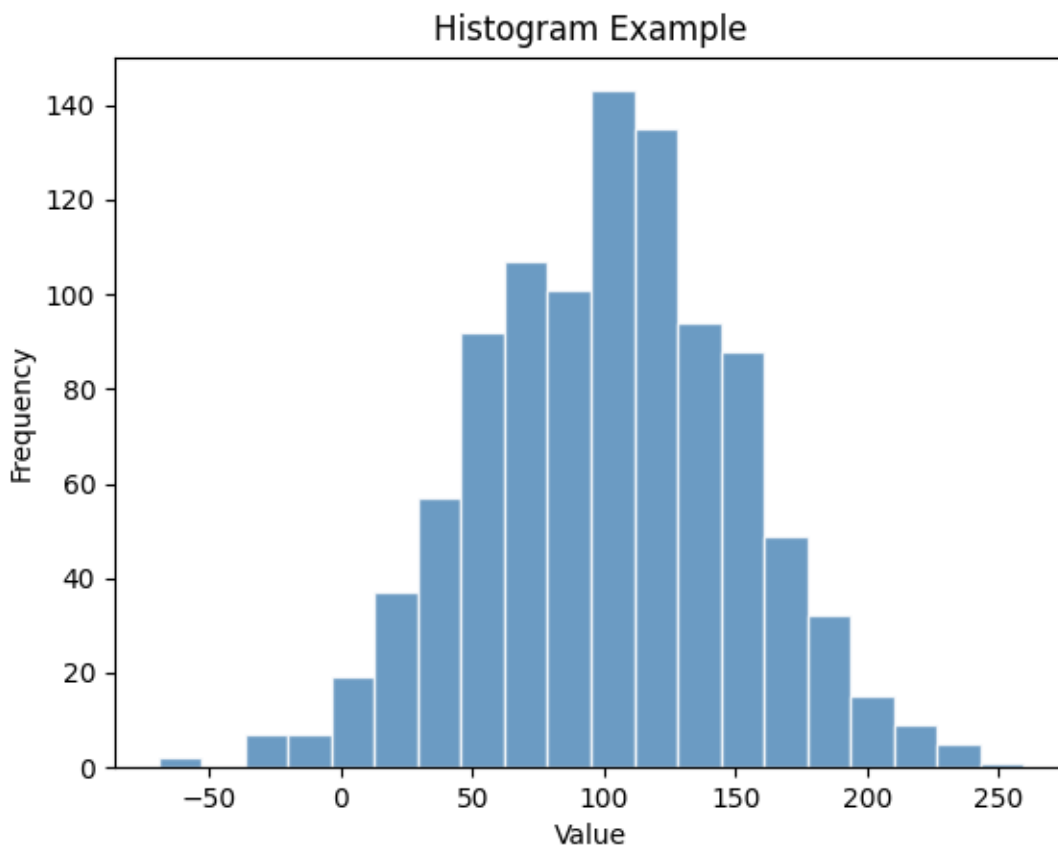
Histograms are the go-to chart for understanding the shape of a distribution — whether it is normal, skewed, bimodal, or uniform. Common applications include exploring age distributions, income distributions, or model prediction errors.

CODE SNIPPET

```
import matplotlib.pyplot as plt
import numpy as np

data = np.random.normal(loc=100, scale=50, size=1000)

plt.hist(data, bins=20, color='steelblue', edgecolor='white', alpha=0.8)
plt.title('Histogram Example')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.show()
```



DESCRIPTION OF PARAMETERS

- `np.random.normal(loc, scale, size)` — Generates 1 000 values from a normal distribution with mean 100 and standard deviation 50.
- `bins` — Number of equal-width intervals into which the data is divided.
- `alpha` — Transparency of the bars (0 = fully transparent, 1 = fully opaque).
- `edgecolor` — Colour of the dividing lines between bars.

2.4 Scatter Plot

DESCRIPTION

A scatter plot represents data as individual dots positioned at (x, y) coordinates. It is used primarily to visualise the relationship or correlation between two numerical variables.

USE CASE

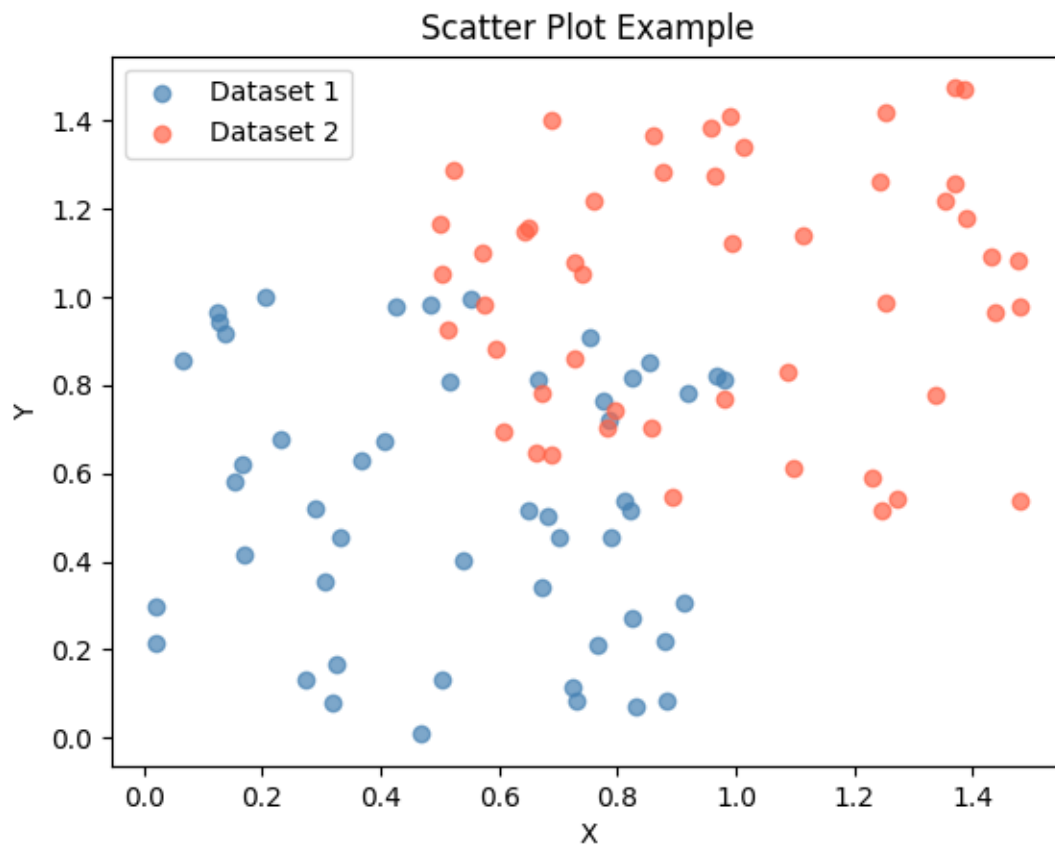
Use scatter plots to investigate correlation, detect outliers, and observe clustering behaviour. Typical uses include plotting height vs weight, advertising spend vs revenue, or feature pairs in machine learning.

CODE SNIPPET

```
import matplotlib.pyplot as plt
import numpy as np

x1, y1 = np.random.rand(50), np.random.rand(50)
x2, y2 = np.random.rand(50) + 0.5, np.random.rand(50) + 0.5

plt.scatter(x1, y1, c='steelblue', label='Dataset 1', alpha=0.7)
plt.scatter(x2, y2, c='tomato', label='Dataset 2', alpha=0.7)
plt.title('Scatter Plot Example')
plt.xlabel('X')
plt.ylabel('Y')
plt.legend()
plt.show()
```



DESCRIPTION OF PARAMETERS

- `plt.scatter(x, y)` — Plots points at the given (x, y) coordinates.
- `c` — Colour of each point; can be a single colour string or an array of values for colour-mapping.
- `label` — Text shown in the legend for this data series.
- `plt.legend()` — Displays the legend using the labels assigned to each scatter call.

2.5 Pie Chart

DESCRIPTION

A pie chart divides a circle into slices proportional to the percentage contribution of each category to a whole. It is effective when you have a small number of categories and want to show part-to-whole relationships.

USE CASE

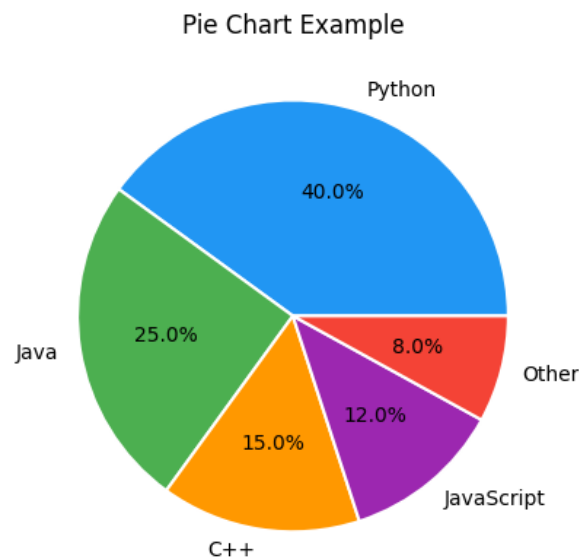
Ideal for showing market share breakdowns, budget allocations, survey results, or any data where the total equals 100%. Limit to five or fewer slices for clarity.

CODE SNIPPET

```
import matplotlib.pyplot as plt

labels = ['Python', 'Java', 'C++', 'JavaScript', 'Other']
sizes = [40, 25, 15, 12, 8]
colors = ['#2196F3', '#4CAF50', '#FF9800', '#9C27B0', '#F44336']

plt.pie(sizes, labels=labels, colors=colors, autopct='%1.1f%%',
        wedgeprops={'linewidth': 1.5, 'edgecolor': 'white'})
plt.title('Pie Chart Example')
plt.show()
```



DESCRIPTION OF PARAMETERS

- **sizes** — A list of values; they do not need to sum to 100 — Matplotlib normalises them automatically.
- **autopct** — Format string that places the percentage label inside each wedge.
- **wedgeprops** — Dictionary of properties applied to each wedge (e.g., border width and colour).

2.6 Area Chart

DESCRIPTION

An area chart fills the region between a line and the x-axis (or between stacked lines), making it easy to see cumulative totals and relative contributions over time.

USE CASE

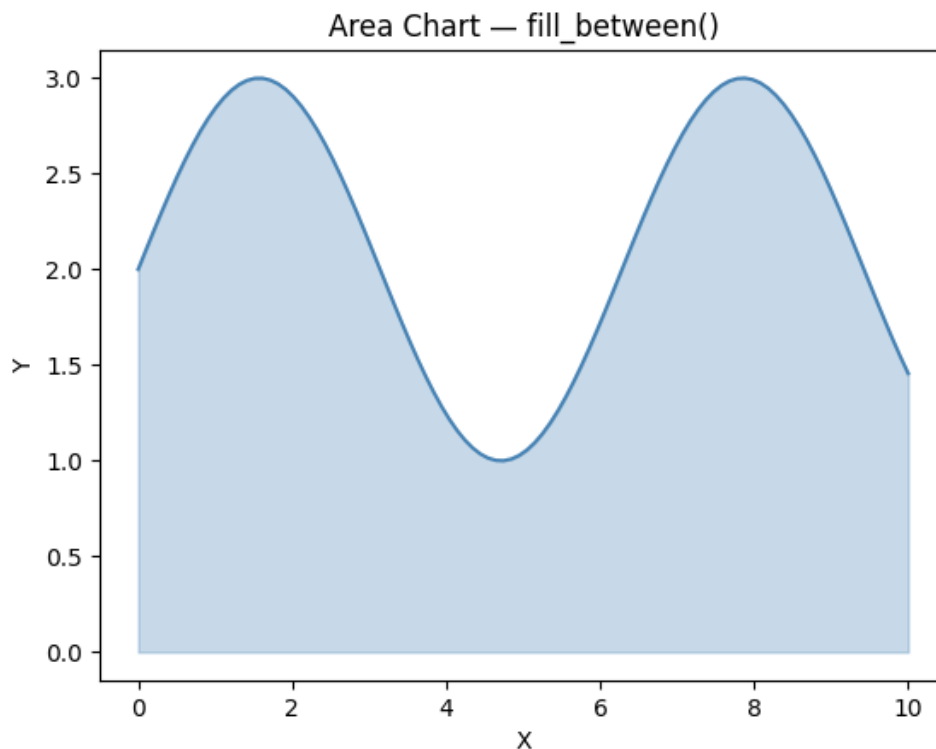
Use area charts to show changes in volume over time and to compare the proportion each category contributes to the total. Common applications include website traffic by channel, revenue by product line, and energy consumption by source.

CODE SNIPPET — `FILL_BETWEEN()`

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 100)
y = np.sin(x) + 2

plt.plot(x, y, color='steelblue')
plt.fill_between(x, y, alpha=0.3, color='steelblue')
plt.title('Area Chart — fill_between()')
plt.xlabel('X')
plt.ylabel('Y')
plt.show()
```

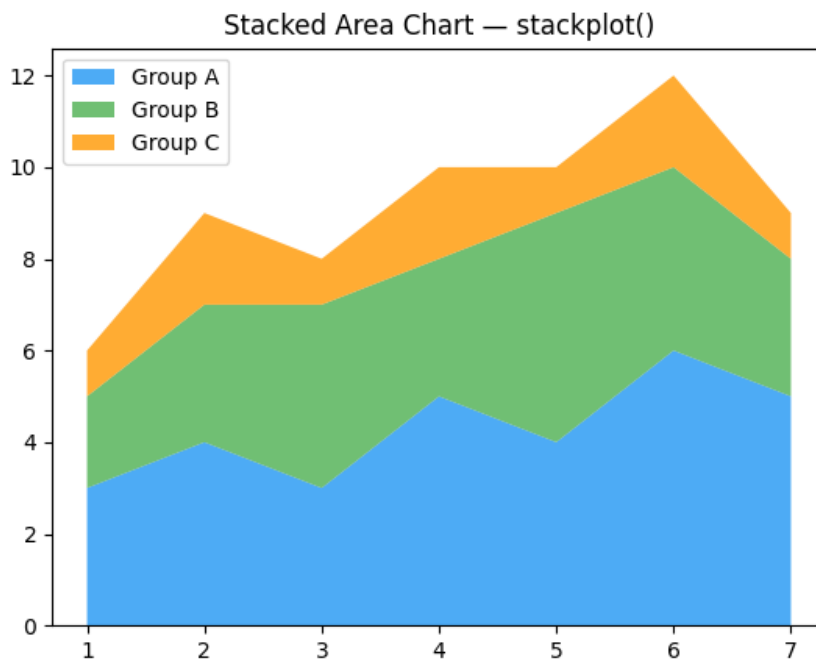


CODE SNIPPET — STACKPLOT()

```
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(1, 8)
y1 = [3, 4, 3, 5, 4, 6, 5]
y2 = [2, 3, 4, 3, 5, 4, 3]
y3 = [1, 2, 1, 2, 1, 2, 1]

plt.stackplot(x, y1, y2, y3,
              labels=['Group A', 'Group B', 'Group C'],
              colors=['#2196F3', '#4CAF50', '#FF9800'], alpha=0.8)
plt.legend(loc='upper left')
plt.title('Stacked Area Chart — stackplot()')
plt.show()
```



DESCRIPTION OF PARAMETERS

- `fill_between(x, y)` — Shades the area between `y` and the baseline (`y=0` by default).
- `stackplot(x, y1, y2, ...)` — Creates a stacked area chart; each `y` series is stacked on top of the previous.
- `labels` — Assigns names to each stacked series for the legend.
- `loc` — Position of the legend (e.g., 'upper left', 'best').

3. SEABORN — GRAPH TYPES

Seaborn organises its plots into five categories: Relational, Categorical, Distribution, Regression, and Matrix. Below are the most important graph types in each category, complete with descriptions, use cases, and code snippets.

3.1 Scatter Plot (Relational)

DESCRIPTION

`sns.scatterplot()` produces a scatter plot with automatic support for colour coding (hue), size encoding, and marker styles — all mapped from columns in a Pandas DataFrame. The resulting chart is immediately more informative than a basic Matplotlib scatter.

USE CASE

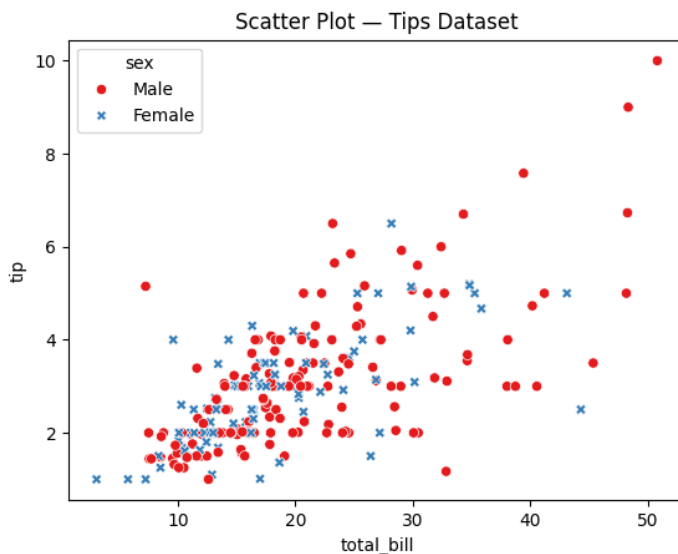
Use Seaborn's scatter plot to explore the relationship between two numeric variables and simultaneously encode a third categorical variable via colour. Ideal for EDA on DataFrame columns.

CODE SNIPPET

```
import seaborn as sns
import matplotlib.pyplot as plt

tips = sns.load_dataset('tips')

sns.scatterplot(data=tips, x='total_bill', y='tip',
                hue='sex', style='sex', palette='Set1')
plt.title('Scatter Plot — Tips Dataset')
plt.show()
```



DESCRIPTION OF PARAMETERS

- `data` — The Pandas DataFrame containing the data.
- `x`, `y` — Column names to use for the horizontal and vertical axes.
- `hue` — Column name whose values control point colour; a legend is added automatically.
- `style` — Column name whose values control point marker shape.
- `palette` — Named colour palette (e.g., 'Set1', 'muted', 'deep').

3.2 Line Plot (Relational)

DESCRIPTION

`sns.lineplot()` draws a line chart with optional confidence intervals computed automatically from repeated observations. This makes it particularly valuable when plotting aggregated time-series data.

USE CASE

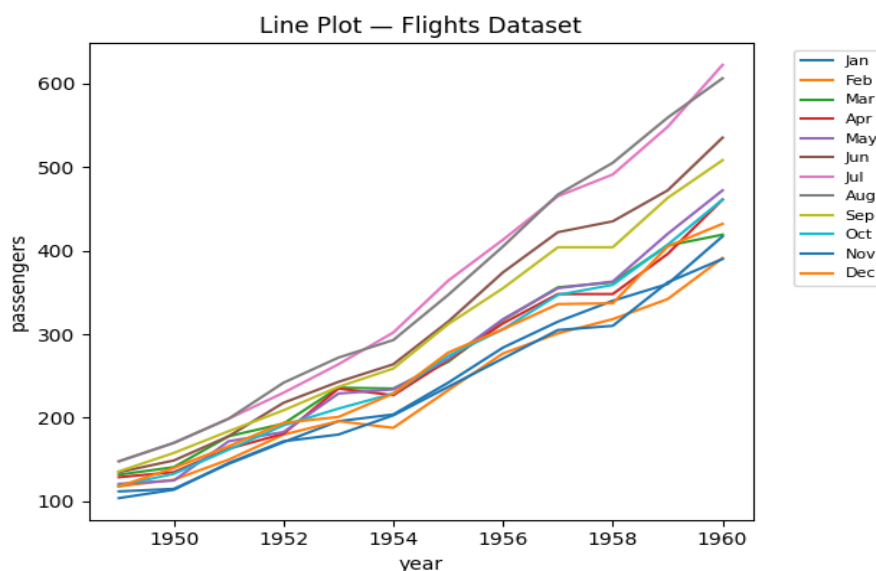
Use when visualising trends over time or across an ordered axis, especially when you want to show mean plus confidence interval bands derived from multiple observations at the same x value.

CODE SNIPPET

```
import seaborn as sns
import matplotlib.pyplot as plt

flights = sns.load_dataset('flights')

sns.lineplot(data=flights, x='year', y='passengers',
             hue='month', palette='tab10')
plt.title('Line Plot — Flights Dataset')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left', fontsize=8)
plt.tight_layout()
plt.show()
```



3.3 Bar Plot (Categorical)

DESCRIPTION

`sns.barplot()` shows the mean (or another estimator) of a numeric variable for each category, with error bars displaying a confidence interval. This is different from Matplotlib's bar chart, which simply plots raw values.

USE CASE

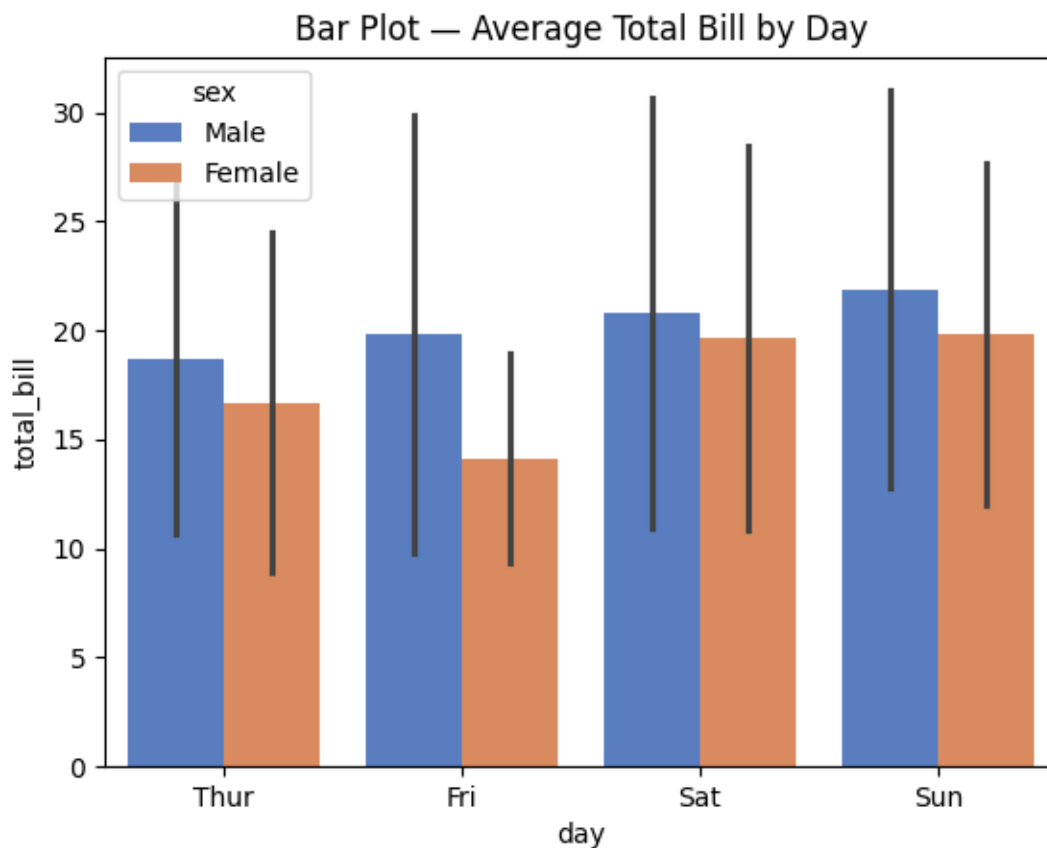
Use when you want to compare the average value of a numeric variable across categories and also want to see the uncertainty around that average. Great for survey data, A/B test results, and grouped comparisons.

CODE SNIPPET

```
import seaborn as sns
import matplotlib.pyplot as plt

tips = sns.load_dataset('tips')

sns.barplot(data=tips, x='day', y='total_bill',
            hue='sex', palette='muted', errorbar='sd')
plt.title('Bar Plot — Average Total Bill by Day')
plt.show()
```



3.4 Count Plot (Categorical)

DESCRIPTION

`sns.countplot()` is a special bar chart that automatically counts the number of observations in each category and plots the result — no aggregation step required from the user.

USE CASE

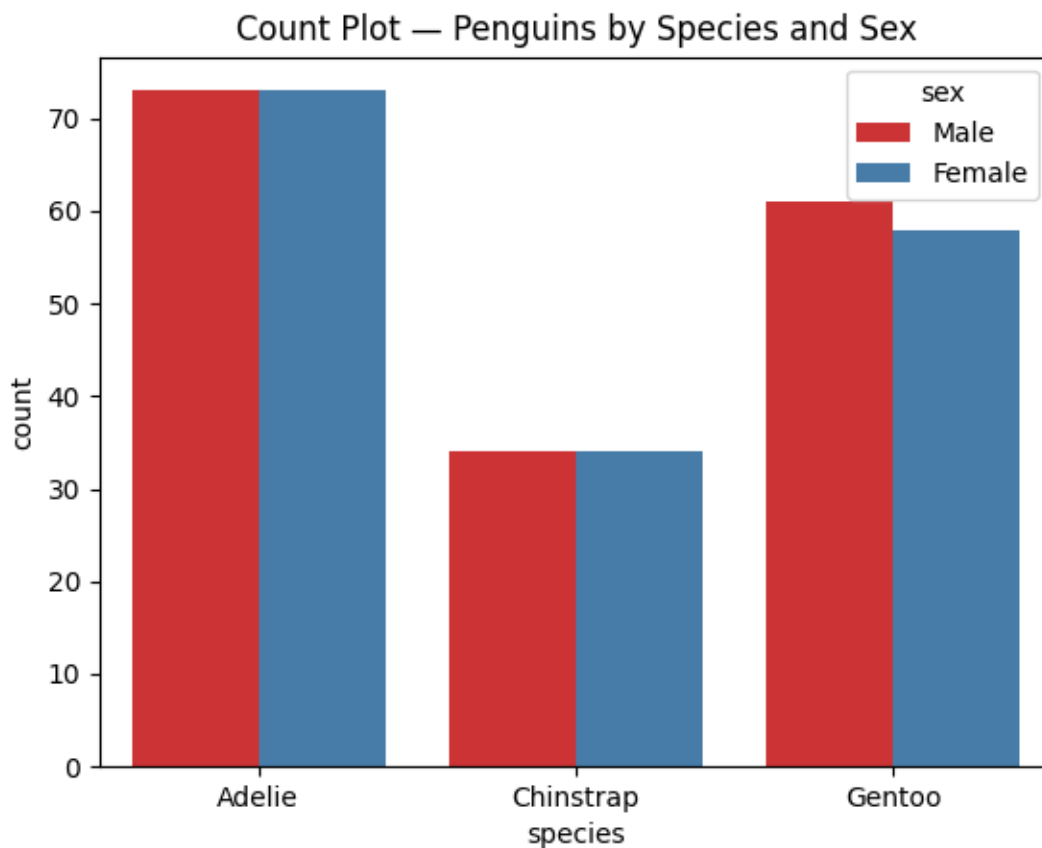
Ideal for categorical frequency analysis: how many passengers are male vs female, how many records fall into each class label, or how many products were sold per category.

CODE SNIPPET

```
import seaborn as sns
import matplotlib.pyplot as plt

penguins = sns.load_dataset('penguins')

sns.countplot(data=penguins, x='species',
              hue='sex', palette='Set1')
plt.title('Count Plot — Penguins by Species and Sex')
plt.show()
```



DESCRIPTION OF PARAMETERS

- x — Categorical column whose unique values define the bars.
- hue — Adds a second grouping; bars within each x category are split by hue colour.
- palette — Colour palette applied to the hue groups.

3.5 Box Plot (Categorical)

DESCRIPTION

A box plot summarises the distribution of a numeric variable using five statistics: minimum, first quartile (Q1), median, third quartile (Q3), and maximum. Data points beyond $1.5 \times \text{IQR}$ from the box are plotted individually as outliers.

USE CASE

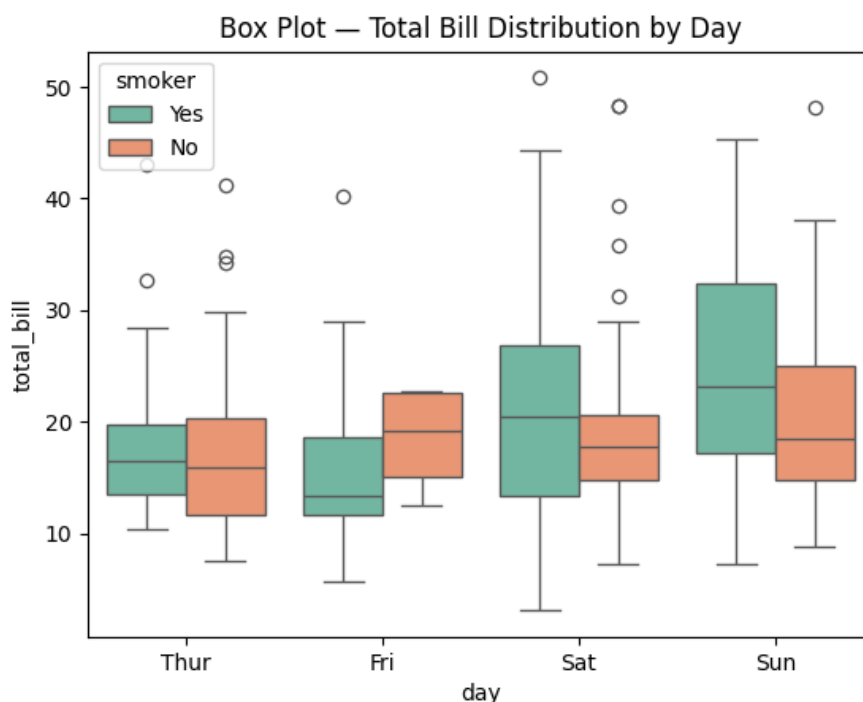
Box plots are excellent for comparing distributions across categories and detecting outliers. Use them to compare salary distributions by job title, test scores by school, or model residuals by input group.

CODE SNIPPET

```
import seaborn as sns
import matplotlib.pyplot as plt

tips = sns.load_dataset('tips')

sns.boxplot(data=tips, x='day', y='total_bill',
            hue='smoker', palette='Set2')
plt.title('Box Plot — Total Bill Distribution by Day')
plt.show()
```



DESCRIPTION OF PARAMETERS

- x — Categorical variable that defines each group along the x-axis.
- y — Numeric variable whose distribution is summarised in the box.
- hue — Optional second grouping; produces side-by-side boxes within each x category.

3.6 Histogram (Distribution)

DESCRIPTION

`sns.histplot()` produces a histogram with optional kernel density estimate (KDE) overlay. Compared to Matplotlib's `hist()`, it integrates with DataFrames and supports hue-based grouping out of the box.

USE CASE

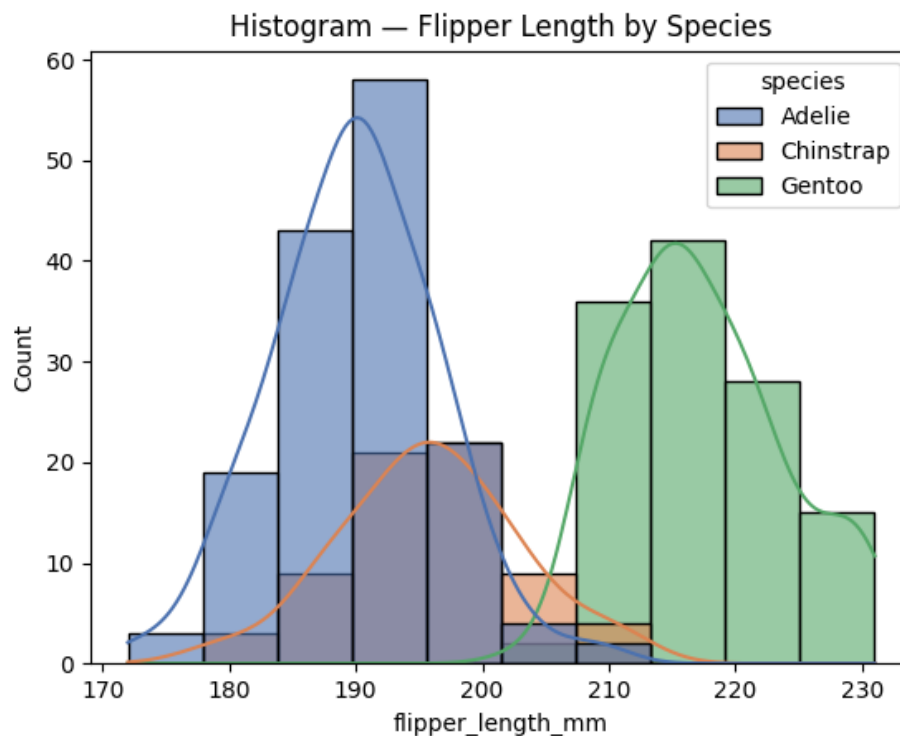
Use to understand the distribution of a single variable and compare distributions across subgroups in a single call.

CODE SNIPPET

```
import seaborn as sns
import matplotlib.pyplot as plt

penguins = sns.load_dataset('penguins')

sns.histplot(data=penguins, x='flipper_length_mm',
             hue='species', kde=True, palette='deep', alpha=0.6)
plt.title('Histogram — Flipper Length by Species')
plt.show()
```



3.7 KDE Plot (Distribution)

DESCRIPTION

A Kernel Density Estimate (KDE) plot draws a smooth probability density curve for a continuous variable. It is analogous to a histogram but without the discretisation caused by bin boundaries.

USE CASE

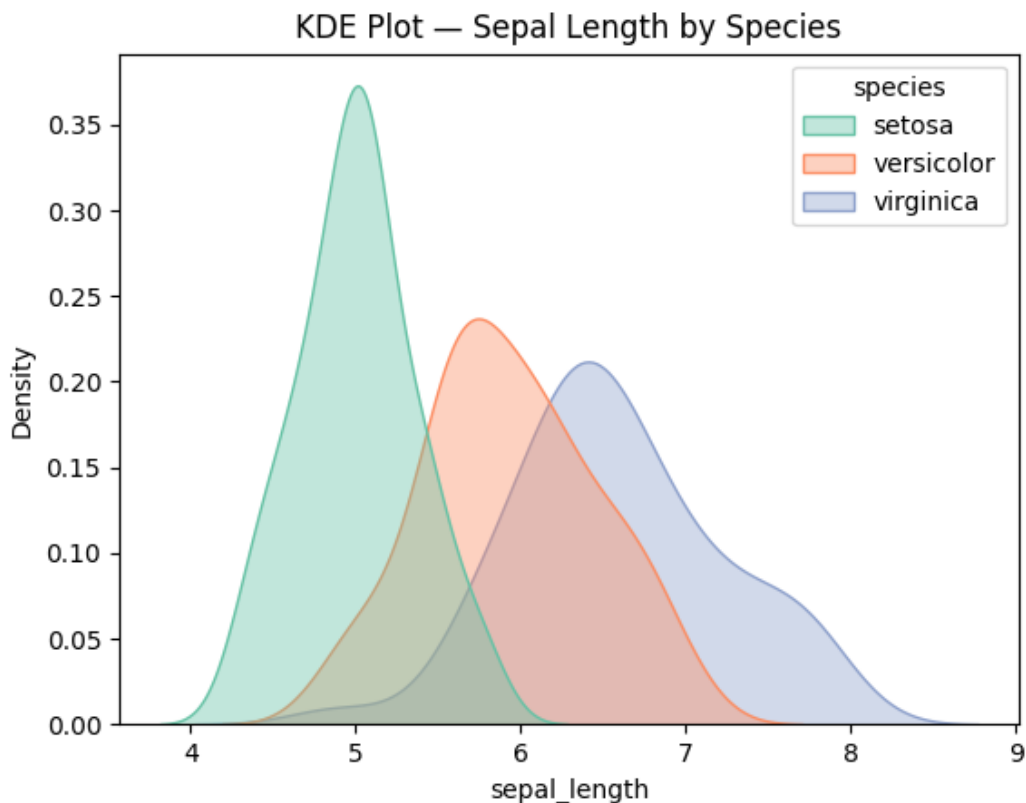
KDE plots are ideal when you want to compare the shape of multiple distributions simultaneously on one axes without the visual clutter of overlapping histograms.

CODE SNIPPET

```
import seaborn as sns
import matplotlib.pyplot as plt

iris = sns.load_dataset('iris')

sns.kdeplot(data=iris, x='sepal_length',
            hue='species', fill=True, palette='Set2', alpha=0.4)
plt.title('KDE Plot — Sepal Length by Species')
plt.show()
```



DESCRIPTION OF PARAMETERS

- `fill=True` — Shades the area under each KDE curve for visual differentiation.
- `alpha` — Controls the transparency of the filled regions so overlapping curves remain visible.

3.8 Heatmap (Matrix)

DESCRIPTION

`sns.heatmap()` visualises a 2-D matrix of values using colour intensity. Cells with higher values receive lighter (or warmer) colours; cells with lower values receive darker (or cooler) colours. It is most often applied to correlation matrices.

USE CASE

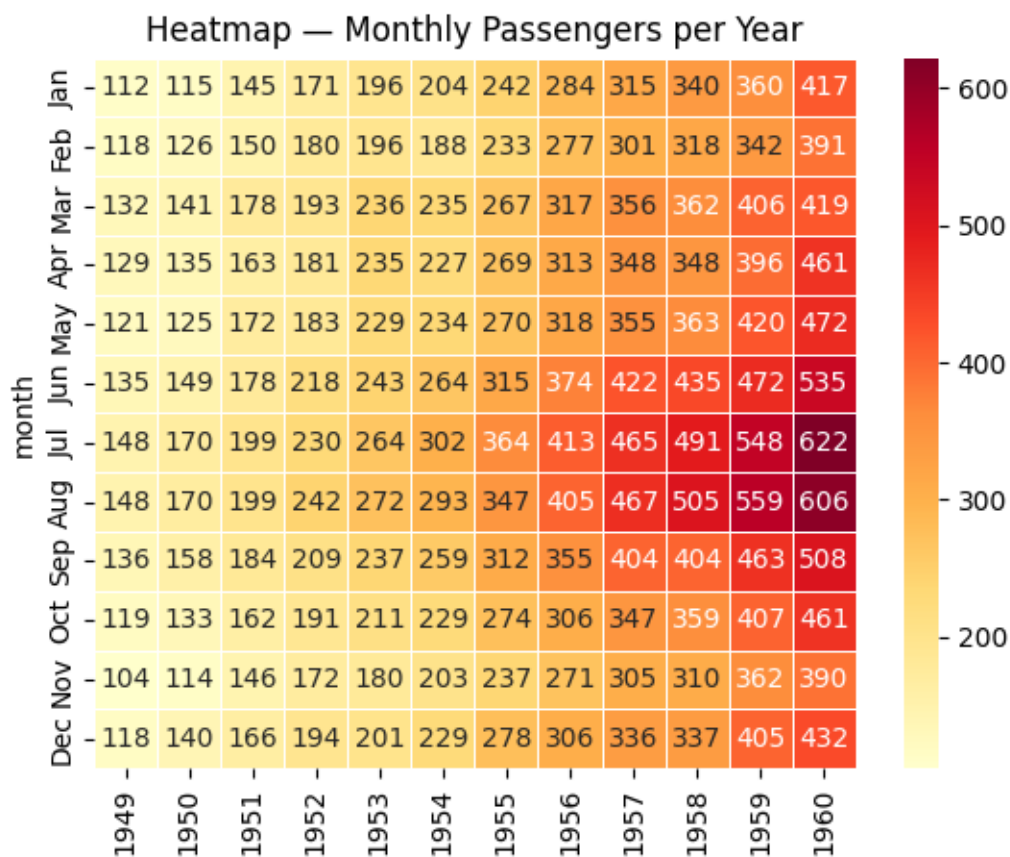
Use heatmaps to reveal correlations between features in a dataset, compare confusion matrix results in a classifier, or display any tabular data where colour-coding aids pattern recognition.

CODE SNIPPET

```
import seaborn as sns
import matplotlib.pyplot as plt

flights = sns.load_dataset('flights')
pivot = flights.pivot(index='month', columns='year',
                      values='passengers')

sns.heatmap(pivot, annot=True, fmt='d', cmap='YlOrRd', linewidths=0.5)
plt.title('Heatmap - Monthly Passengers per Year')
plt.show()
```



DESCRIPTION OF PARAMETERS

- `annot=True` — Writes the numeric value of each cell inside the cell.
- `fmt` — Format string for the annotation (e.g., 'd' for integer, '.2f' for two decimal places).
- `cmap` — Colour map; 'YlOrRd' goes from yellow (low) through orange to red (high).
- `linewidths` — Width of the lines separating cells.

3.9 Regression Plot (Regression)

DESCRIPTION

`sns.regplot()` draws a scatter plot of two variables and overlays a linear regression line with a shaded 95% confidence interval. It makes the linear trend between variables immediately visible without any additional fitting code.

USE CASE

Use regression plots to visually assess the linear relationship between two numeric variables, check regression assumptions, or present model fit to a non-technical audience.

CODE SNIPPET

```
import seaborn as sns
import matplotlib.pyplot as plt

tips = sns.load_dataset('tips')

sns.regplot(data=tips, x='total_bill', y='tip',
            scatter_kws={'alpha': 0.5, 'color': 'steelblue'},
            line_kws={'color': 'tomato', 'linewidth': 2})
plt.title('Regression Plot — Tip vs Total Bill')
plt.show()
```



DESCRIPTION OF PARAMETERS

- `scatter_kws` — Dictionary of keyword arguments passed directly to the underlying scatter plot.
- `line_kws` — Dictionary of keyword arguments applied to the regression line.
- Confidence interval — Shaded band around the line; controlled with `ci=` parameter (default 95).

4. LIBRARY COMPARISON

Choosing between Matplotlib and Seaborn is not always an either-or decision — many practitioners use both within the same project. The table below and the narrative that follows highlight the key trade-offs across four dimensions: ease of use, customisation, interactivity, and performance on large datasets.

Dimension	Matplotlib	Seaborn
Ease of Use	Moderate — requires more lines of code for aesthetically pleasing results; steeper learning curve.	High — minimal code for complex statistical charts; sensible defaults out of the box.
Customisation	Excellent — full control over every visual property through the object-oriented API.	Good — inherits Matplotlib customisation but high-level API hides some low-level options.
Interactivity	Limited in static mode; interactive backends (Qt, TkAgg) available but require extra setup.	None natively — relies on Matplotlib backends; use Plotly or Bokeh for interactive requirements.
Performance (Large Data)	Handles millions of points; performance depends on the backend (Agg is fastest).	Adds a small overhead for statistical computations; underlying Matplotlib rendering is the bottleneck.
DataFrame Integration	Indirect — requires explicit extraction of columns as NumPy arrays.	Seamless — pass the DataFrame directly; column names used as axis labels automatically.
Statistical Features	None built-in — must compute statistics (means, CI) separately before plotting.	Rich — confidence intervals, regression lines, kernel densities computed automatically.
Default Aesthetics	Basic MATLAB-style defaults; requires manual theming.	Polished themes (darkgrid, whitegrid, etc.) and colour palettes ready to use.
Community & Docs	Very large — oldest library with the most examples, tutorials, and Stack Overflow answers.	Large — rapidly growing; official gallery and documentation are excellent.

4.1 Ease of Use

Seaborn wins decisively for beginners and for analysts who need results quickly. A publication-ready scatter plot coloured by a categorical variable requires a single `sns.scatterplot()` call versus multiple `plt.scatter()` calls, manual legend creation, and colour-cycling in Matplotlib.

However, Matplotlib's syntax becomes natural with practice, and its conceptual model (Figure → Axes → Artists) is well worth learning because it underpins every other Python plotting library.

4.2 Customisation

Matplotlib's object-oriented API provides surgical control. You can move a tick label by a single pixel, change the cap style of a specific line end, or embed a chart inside a custom GUI widget — things that Seaborn's high-level API does not expose. For reports where exact pixel-perfect output matters, Matplotlib is the right tool.

Seaborn supports customisation via `sns.set_theme()`, `sns.set_palette()`, and `set_style()`, and you can always drop down to the underlying Matplotlib Axes object (returned by every Seaborn plotting function) to make low-level adjustments.

4.3 Interactivity

Neither library is designed for interactive web-based visualisations. If interactivity is a primary requirement, consider Plotly (which produces HTML/JavaScript charts) or Bokeh (a full interactive charting server). Both can be used alongside Matplotlib and Seaborn in the same analysis notebook.

4.4 Performance with Large Datasets

Both libraries rely on Matplotlib's rendering pipeline. For datasets exceeding one million points, the default Agg backend (used in scripts and notebooks) is the fastest option but still has limits. For very large datasets, consider:

- **Datashader** — Rasterises billions of points into pixel grids before passing to Matplotlib.
- **Sampling** — Plot a representative random sample (10 000–100 000 points) when exact position of every point is not needed.
- **Aggregation** — Replace a scatter plot of 1 M points with a 2-D histogram (`plt.hist2d` or `sns.histplot(bins=...)`) to show density instead of individual points.

4.5 When to Use Which

Choose Matplotlib when...

You need publication-quality figures with precise control over every element. You are building subplots, custom layouts, or non-standard chart types. You need to embed charts in a GUI application or export to SVG/PDF for a print journal.

Choose Seaborn when...

You are doing exploratory data analysis on a Pandas DataFrame. You want automatic confidence intervals, regression lines, or KDE overlays. You need aesthetically polished charts with minimal code for a presentation or dashboard.

Use both together when...

You generate the base chart with Seaborn for its statistical features and default aesthetics, then refine titles, ticks, font sizes, and annotations using Matplotlib's Axes object. This combination is the most common pattern in professional data science notebooks.

5. CONCLUSION

Matplotlib and Seaborn form a powerful and complementary pair in the Python data visualisation ecosystem. Matplotlib provides the foundational rendering engine and maximum flexibility, while Seaborn adds an intuitive, statistics-aware layer that dramatically reduces boilerplate and improves default aesthetics.

The strategic choice between the two — or the decision to combine them — should be driven by the task at hand:

- For quick EDA and statistical insight, reach for Seaborn.
- For custom, publication-ready figures, reach for Matplotlib.
- For interactive web visualisations, look beyond both to Plotly or Bokeh.

By mastering both libraries and understanding when to apply each, you will have the tools to communicate data effectively at every stage of the data science workflow — from initial exploration to final presentation.

*DATA VISUALIZATION LIBRARIES DOCUMENTATION
(Matplotlib & Seaborn)*

*Submitted for : ShadowFox Data Science Internship
(Task 1 – Beginner Level)*

Prepared By: Tuba Mariyam Khateeb

Course: Data Science Internship

Date: May 2026

— End of Documentation—